

Nerra Network

YouTube Data API v3 — Upload Script & Daily Upload Volume

Prepared for	YouTube API Services review — follow-up to our submission
GCP project number	141610975484
API client	Nerra Network Publisher (internal, server-side automation)
Operator / contact	Patrick Novak · patrick@planetterrian.com
Date	May 22, 2026

Your two questions, answered

1. Complete script demonstrating how the client uploads videos.

The full upload module (`engine/youtube.py`) is reproduced verbatim in Section 2 and attached as a `.py` file. It performs `videos.insert` (resumable upload), `thumbnails.set`, `captions.insert`, and `playlistItems.insert`. Section 1 walks the flow step by step and shows the per-episode orchestration code.

2. Number of videos uploaded per day via the API.

Today: 4 videos/day — two shows are enabled (Tesla Shorts Time and Models & Agents for Beginners), each uploading one long-form video plus one Shorts clip. **At full rollout the request supports up to 22 videos/day** (11 shows $\times$ 2), plus a temporary historical backfill. See Section 3.

1. How the API client uploads videos

The client is an internal, server-side Python automation that runs unattended on a daily GitHub Actions cron. No end user interacts with it; it only ever writes to the two YouTube channels owned by the operator. For each enabled show, one run does the following:

- 1. Authenticate.** `get_channel_credentials_from_env()` builds OAuth 2.0 credentials from a long-lived refresh token (minted once via a bootstrap script, stored as a GitHub secret). One token per owned channel.
- 2. Render the media.** The pipeline renders a 16:9 long-form MP4 (cover + audio waveform + image slideshow + burned-in captions) and a 9:16 Shorts clip.
- 3. Upload the long-form video.** `upload_video()` calls `youtube.videos().insert()` as a resumable (chunked) upload, with `status.containsSyntheticMedia = true` and a written AI disclosure in the description.
- 4. Set the thumbnail.** Within the same call, `youtube.thumbnails().set()` attaches a 1280×720 thumbnail.
- 5. Attach a caption track.** `upload_caption_track()` calls `youtube.captions().insert()` with the episode's SRT (long-form only).
- 6. Add to the show playlist.** `add_video_to_playlist()` calls `youtube.playlistItems().insert()` to file the video in the show's podcast playlist.

7. Repeat for the Shorts clip (steps 3, 4, 6). Captions are long-form only.

Per-episode orchestration (excerpt from run_show.py)

This is the exact call sequence per episode — credential load, the long-form upload with its playlist add and caption track, then the Shorts upload. Media-rendering and error-handling lines are elided with comments for readability:

```
# ---- run_show.py : _publish_youtube() (per-episode upload) ----
from engine.youtube import (
    add_video_to_playlist,
    get_channel_credentials_from_env,
    upload_video,
)

credentials = get_channel_credentials_from_env(config.youtube.channel)
if credentials is None:
    logger.warning(
        "YouTube credentials missing for channel=%s - skipping upload.",
        config.youtube.channel,
    )
    return result

# ... per-episode media is rendered here: a 16:9 long-form MP4
# (cover + audio waveform + image slideshow + burned-in captions)
# and a 9:16 vertical Shorts clip. Omitted for brevity. ...

# ---- Long-form ----
long_url = ""
if config.youtube.publish_long_form:
    try:
        build_long_form_video(
            final_mp3, cover_path, long_video_path,
            scene_paths=long_scene_paths if len(long_scene_paths) >= 2 else None,
            subtitles_path=srt_path,
            show_name=config.name,
        )
        meta = build_long_form_metadata(
            config,
            episode_num=episode_num,
            today_str=today_str,
            hook=hook,
            digest_text=digest_text,
            audio_url=audio_url,
            chapters_path=chapters_path if chapters_path.exists() else None,
            photo_attribution=pexels_attribution,
        )
        upload = upload_video(
            long_video_path,
            credentials=credentials,
            title=meta["title"],
            description=meta["description"],
            tags=meta["tags"],
            category_id=meta["category_id"],
```

```

        default_language=meta["default_language"],
        privacy_status=config.youtube.privacy_status,
        thumbnail_path=thumbnail_path,
    )
    long_url = upload.watch_url
    result["long_url"] = long_url
    playlist_id = (
        getattr(config.youtube, "podcast_playlist_id", None) or ""
    ).strip()
    if not playlist_id:
        logger.info(
            "Podcast playlist ID empty – skipping playlist add."
        )
    else:
        add_video_to_playlist(
            credentials=credentials,
            video_id=upload.video_id,
            playlist_id=playlist_id,
        )
    # Upload the SRT as a real caption track. This is on top of
    # the burned-in captions – gives YouTube the CC button +
    # auto-translation + accessibility search. Best-effort:
    # failures are logged and the run continues.
    if srt_path and srt_path.exists():
        from engine.youtube import upload_caption_track
        lang_code = (config.youtube.default_language or "en").lower()
        track_name = "English" if lang_code == "en" else (
            "Русский" if lang_code == "ru" else lang_code.upper()
        )
        upload_caption_track(
            credentials=credentials,
            video_id=upload.video_id,
            srt_path=srt_path,
            language=lang_code,
            name=track_name,
        )

    # (long-form upload errors are caught and recorded to metrics) ...

# ---- Shorts ----
if config.youtube.publish_shorts and should_upload_shorts_today(
    config, episode_num=episode_num,
):
    try:
        short_offset = resolve_shorts_start_offset(
            config,

```

```

        chapters_path if chapters_path.exists() else None,
        audio_duration=_ep_duration,
    )
    duration = float(config.youtube.short_duration_seconds or 55.0)
    build_short_video(
        final_mp3, cover_path, short_video_path,
        start_offset=short_offset,
        duration=duration,
        hook=hook or None,
        scene_paths=short_scene_paths if len(short_scene_paths) >= 2 else None,
        show_name=config.name,
    )
    meta = build_short_metadata(
        config,
        episode_num=episode_num,
        today_str=today_str,
        hook=hook,
        long_form_url=long_url,
    )
    short_thumb = (
        short_thumbnail_path
        if short_thumbnail_path and Path(short_thumbnail_path).exists()
        else thumbnail_path
    )
    short_upload = upload_video(
        short_video_path,
        credentials=credentials,
        title=meta["title"],
        description=meta["description"],
        tags=meta["tags"],
        category_id=meta["category_id"],
        default_language=meta["default_language"],
        privacy_status=config.youtube.privacy_status,
        thumbnail_path=short_thumb,
    )
    result["short_url"] = short_upload.watch_url
    playlist_id = (
        getattr(config.youtube, "podcast_playlist_id", None) or ""
    ).strip()
    if not playlist_id:
        logger.info(
            "Podcast playlist ID empty – skipping playlist add."
        )
    else:
        add_video_to_playlist(
            credentials=credentials,

            video_id=short_upload.video_id,
            playlist_id=playlist_id,
        )

# (Shorts upload errors are likewise caught and recorded) ...

```

2. The complete upload script (engine/youtube.py)

Reproduced verbatim below and attached as `Nerra_Network_youtube_upload_script.py`. This single module is the entirety of the client's YouTube write path: credential construction, the resumable `videos.insert` upload with thumbnail, the caption-track upload, and the playlist add.

```
"""YouTube publishing wrapper for the podcast pipeline.

Provides:

- :func:`upload_video` - resumable video upload + thumbnail set in one
  call. Mirrors the ``post_to_x()`` style: explicit credentials in,
  watch URL out.
- :func:`build_oauth_credentials` - turn a long-lived refresh token
  into a usable :class:`google.oauth2.credentials.Credentials`. Used
  in production (CI) where we cannot run a browser flow.
- :func:`get_channel_credentials_from_env` - convenience wrapper that
  reads the four ``YOUTUBE_*`` env vars used by the workflow.

Every upload sets ``status.containsSyntheticMedia=True`` because all
Nerra Network episodes use AI voice synthesis (Grok TTS network-wide
since May 2026; ElevenLabs is retained as an emergency rollback only).
This is the API
field YouTube introduced in October 2024 for AI/A&S disclosure; setting
it via the API renders the same "Altered or synthetic content" label
the Studio UI applies, and is required for monetization-eligible AI
audio uploads.

Quotas
-----
``videos.insert`` costs **1,600 quota units** per upload. The default
project quota is 10,000 units/day. ``thumbnails.set`` adds another
50 units. Plan accordingly - see ``docs/youtube_setup.md``.
"""

from __future__ import annotations

import logging
import os
from dataclasses import dataclass
from pathlib import Path
from typing import List, Optional

from tenacity import (
    retry,
    retry_if_exception,
    stop_after_attempt,
    wait_exponential,
)

logger = logging.getLogger(__name__)
```

```

# OAuth scopes required for upload + thumbnail set + channel read +
# caption track upload (`captions.insert` needs force-ssl).
YOUTUBE_SCOPES = [
    "https://www.googleapis.com/auth/youtube.upload",
    "https://www.googleapis.com/auth/youtube",
    "https://www.googleapis.com/auth/youtube.force-ssl",
]

# Google's OAuth token endpoint. Held as a constant rather than an env
# var so misconfigured deployments fail loudly at import time, not
# halfway through a run.
GOOGLE_TOKEN_URI = "https://oauth2.googleapis.com/token"

# -----
# Credential helpers
# -----

def build_oauth_credentials(
    *,
    client_id: str,
    client_secret: str,
    refresh_token: str,
):
    """Return a refreshable :class:`Credentials` for the YouTube API.

    The refresh token is what we mint once via the bootstrap script
    (`scripts/youtube_oauth_bootstrap.py`) and store as a GitHub
    secret. Google's library automatically swaps it for a short-lived
    access token on every API call.
    """
    from google.oauth2.credentials import Credentials

    if not client_id or not client_secret:
        raise ValueError(
            "YouTube OAuth client_id/client_secret are required"
        )
    if not refresh_token:
        raise ValueError("YouTube OAuth refresh_token is required")

    return Credentials(
        token=None,
        refresh_token=refresh_token,
        token_uri=GOOGLE_TOKEN_URI,
        client_id=client_id,
    )

```

```

        client_secret=client_secret,
        scopes=YOUTUBE_SCOPES,
    )

def get_channel_credentials_from_env(channel: str = "en"):
    """Read the four ``YOUTUBE_*`` env vars and return Credentials.

    *channel* picks which refresh token to load: ``"en"`` →
    ``YOUTUBE_REFRESH_TOKEN_EN``, ``"ru"`` →
    ``YOUTUBE_REFRESH_TOKEN_RU``. Returns ``None`` (not an exception)
    if any value is missing – the caller decides whether to skip the
    upload or fail the run.
    """
    client_id = os.getenv("YOUTUBE_CLIENT_ID", "").strip()
    client_secret = os.getenv("YOUTUBE_CLIENT_SECRET", "").strip()
    suffix = "RU" if channel.lower() == "ru" else "EN"
    refresh_token = os.getenv(f"YOUTUBE_REFRESH_TOKEN_{suffix}", "").strip()

    if not all([client_id, client_secret, refresh_token]):
        logger.info(
            "YouTube credentials not fully set for channel=%s – skipping",
            channel,
        )
        return None

    return build_oauth_credentials(
        client_id=client_id,
        client_secret=client_secret,
        refresh_token=refresh_token,
    )

# -----
# Upload
# -----

# Errors that warrant a retry. Pulled in lazily inside
# ``_should_retry_http`` so the module imports cleanly even when the
# google libraries are missing (e.g. in pure-unit-test environments).
def _is_retryable_http_error(exc: BaseException) -> bool:
    try:
        from googleapiclient.errors import HttpError
    except ImportError: # pragma: no cover
        return False
    if not isinstance(exc, HttpError):

```

```

        return False
    status = getattr(getattr(exc, "resp", None), "status", 0)
    return status in (429, 500, 502, 503, 504)

def _build_video_body(
    *,
    title: str,
    description: str,
    tags: List[str],
    category_id: int,
    default_language: str,
    privacy_status: str,
    contains_synthetic_media: bool,
    made_for_kids: bool,
) -> dict:
    """Construct the request body for `youtube.videos().insert()`."""
    return {
        "snippet": {
            "title": title,
            "description": description,
            "tags": tags,
            "categoryId": str(category_id),
            "defaultLanguage": default_language,
            "defaultAudioLanguage": default_language,
        },
        "status": {
            "privacyStatus": privacy_status,
            "selfDeclaredMadeForKids": made_for_kids,
            "containsSyntheticMedia": contains_synthetic_media,
            # Skip YouTube's gradual rollout to subscribers' homepages
            # (we have RSS + X for that). False = publish to subs feed.
            "publishToSubscriptions": True,
        },
    }

def _is_retryable_upload_error(exc: BaseException) -> bool:
    """Retry transient failures only – not 400 invalidDescription / auth."""
    if _is_retryable_http_error(exc):
        return True
    try:
        from googleapiclient.errors import HttpError
    except ImportError: # pragma: no cover
        return False
    if isinstance(exc, HttpError):

```

```

        status = getattr(getattr(exc, "resp", None), "status", 0)
        return status in (429, 500, 502, 503, 504)
    return False

@retry(
    retry=retry_if_exception(_is_retryable_upload_error),
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=2, min=4, max=30),
    reraise=True,
)
def _execute_resumable_upload(insert_request) -> dict:
    """Drive the chunked upload loop until the API returns a video resource.

    Tenacity wraps the whole call so a hard 5xx during chunk transfer
    gets retried from scratch with backoff. ``next_chunk`` itself does
    not retry – it returns ``(status, response)`` per chunk.
    """
    response = None
    while response is None:
        status, response = insert_request.next_chunk()
        if status:
            logger.info("YouTube upload progress: %d%%",
                        int(status.progress() * 100))
    return response

def upload_video(
    video_path: Path,
    *,
    credentials,
    title: str,
    description: str,
    tags: List[str],
    category_id: int,
    default_language: str = "en",
    privacy_status: str = "public",
    thumbnail_path: Optional[Path] = None,
    contains_synthetic_media: bool = True,
    made_for_kids: bool = False,
) -> "UploadResult":
    """Upload a single video and (optionally) its custom thumbnail.

    Parameters
    -----
    video_path:

```

```

    Local MP4 to upload.
credentials:
    Refreshable :class:`google.oauth2.credentials.Credentials` –
    usually from :func:`get_channel_credentials_from_env`.
title, description, tags, category_id, default_language:
    Snippet metadata. ``description`` may include the YouTube
    chapter block; YouTube parses it automatically when the format
    is right (first stamp ``0:00``, ≥ 2 stamps).
privacy_status:
    ``"public"`` | ``"unlisted"`` | ``"private"``.
thumbnail_path:
    Optional 1280x720 JPEG/PNG. Skipped silently when ``None``.
contains_synthetic_media:
    Sets ``status.containsSyntheticMedia``. Defaults to ``True``
    because every Nerra Network episode uses synthesized voice.
made_for_kids:
    ``status.selfDeclaredMadeForKids``. Default ``False``; flip per
    show in the YAML if a show is targeted at < 13.

Returns
-----
UploadResult
    Bundle of ``video_id`` and the canonical ``watch_url``
    (``https://www.youtube.com/watch?v=...``). Callers that only
    want the URL should use ``result.watch_url``.
"""
if not video_path.exists():
    raise FileNotFoundError(f"video not found: {video_path}")

# Lazy imports so ``import engine.youtube`` works in environments
# where the google libs aren't installed (e.g. fresh CI before
# ``pip install -r requirements.txt``).
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

youtube = build("youtube", "v3", credentials=credentials,
               cache_discovery=False)

body = _build_video_body(
    title=title,
    description=description,
    tags=tags,
    category_id=category_id,
    default_language=default_language,
    privacy_status=privacy_status,
    contains_synthetic_media=contains_synthetic_media,

```

```

        made_for_kids=made_for_kids,
    )

    media = MediaFileUpload(
        str(video_path),
        mimetype="video/mp4",
        chunksize=8 * 1024 * 1024,
        resumable=True,
    )

    logger.info(
        "Uploading to YouTube: %s (%.1f MiB, privacy=%s, AI-disclosed)",
        video_path.name,
        video_path.stat().st_size / (1024 * 1024),
        privacy_status,
    )

    insert_request = youtube.videos().insert(
        part="snippet,status",
        body=body,
        media_body=media,
    )

    response = _execute_resumable_upload(insert_request)
    video_id = response.get("id")
    if not video_id:
        raise RuntimeError(
            f"YouTube upload returned no video id: {response!r}"
        )
    watch_url = f"https://www.youtube.com/watch?v={video_id}"
    logger.info("Uploaded: %s", watch_url)

    if thumbnail_path and thumbnail_path.exists():
        try:
            # Wrap in MediaFileUpload with explicit ``mimetype`` to silence
            # the ``googleapiclient.discovery: media_mime_type`` argument
            # not specified; trying to auto-detect`` warning that fires
            # on every YouTube run otherwise. The thumbnail is a JPEG by
            # convention (``engine.video`` writes ``*_thumb.jpg``); fall
            # back to the suffix-derived value if the operator swaps in
            # a PNG / WebP later.
            _ext = thumbnail_path.suffix.lower()
            mime = (
                "image/png" if _ext == ".png"
                else "image/webp" if _ext == ".webp"
                else "image/jpeg"
            )

```

```

    )
    thumb_media = MediaFileUpload(
        str(thumbnail_path), mimetype=mime, resumable=False,
    )
    youtube.thumbnails().set(
        videoId=video_id,
        media_body=thumb_media,
    ).execute()
    logger.info("Thumbnail set for %s", video_id)
except Exception as exc: # pragma: no cover - thumbnail is best-effort
    logger.warning("Thumbnail upload failed (non-fatal): %s", exc)

return UploadResult(video_id=video_id, watch_url=watch_url)

@dataclass
class UploadResult:
    """Return value of :func:`upload_video`.

    Exposes both the bare ``video_id`` (needed for follow-up API calls
    like ``playlistItems.insert``) and the canonical ``watch_url`` that
    ends up in RSS feeds and X posts.
    """
    video_id: str
    watch_url: str

def add_video_to_playlist(
    *,
    credentials,
    video_id: str,
    playlist_id: str,
) -> bool:
    """Append a video to a playlist via the YouTube Data API.

    Returns ``True`` on success, ``False`` on any 4xx (we don't own
    the playlist, video is private and not viewable, etc.) or 5xx
    (which we don't retry – playlist membership is best-effort, not
    the upload itself).

    Costs 50 quota units per call.
    """
    from googleapiclient.discovery import build
    from googleapiclient.errors import HttpError

    youtube = build(

```

```

        "youtube", "v3", credentials=credentials, cache_discovery=False,
    )
    body = {
        "snippet": {
            "playlistId": playlist_id,
            "resourceId": {
                "kind": "youtube#video",
                "videoId": video_id,
            },
        },
    }
}
try:
    youtube.playlistItems().insert(part="snippet", body=body).execute()
except HttpError as exc:
    status = getattr(getattr(exc, "resp", None), "status", "?")
    logger.warning(
        "Failed to add video %s to playlist %s (HTTP %s): %s",
        video_id, playlist_id, status, exc,
    )
    return False
logger.info(
    "added video %s to playlist %s", video_id, playlist_id,
)
return True

def upload_caption_track(
    *,
    credentials,
    video_id: str,
    srt_path: Path,
    language: str = "en",
    name: str = "English",
    is_draft: bool = False,
) -> bool:
    """Upload a caption track (SRT) to a YouTube video via ``captions.insert``.

    This is on top of the burned-in captions we render into the video
    pixels – uploading a real caption track lets YouTube:

    - Show the CC button so viewers can toggle captions
    - Auto-translate to other languages
    - Surface the captions in YouTube search
    - Serve them to screen readers / accessibility tools

    Costs **400 quota units** per call.

```

```

Parameters
-----
credentials:
    Refreshable :class:`Credentials` (same one used for upload).
video_id:
    The video ID returned by :func:`upload_video`.
srt_path:
    Path to the SRT subtitle file to upload (output of
    :func:`engine.captions.transcript_to_srt`).
language:
    BCP-47 language code (``"en"`` , ``"ru"`` , etc.). Must match
    what the captions actually contain.
name:
    Human-readable track name shown in YouTube's CC menu (e.g.
    ``"English"`` , ``"Русский"`` ).
is_draft:
    ``True`` keeps the track as a draft (operator must publish in
    Studio); ``False`` (default) publishes immediately.

Returns
-----
bool
    ``True`` on success; ``False`` if the SRT is missing or the
    API call fails (logged, never raised – caption upload is
    best-effort and must not crash a run).
"""
if not srt_path or not srt_path.exists():
    logger.info("No SRT file at %s – skipping caption track upload",
               srt_path)
    return False
if not video_id:
    logger.warning("No video ID – skipping caption track upload")
    return False

from googleapiclient.discovery import build
from googleapiclient.errors import HttpError
from googleapiclient.http import MediaFileUpload

youtube = build(
    "youtube", "v3", credentials=credentials, cache_discovery=False,
)
body = {
    "snippet": {
        "videoId": video_id,
        "language": language,

```

```

        "name": name,
        "isDraft": is_draft,
    },
}
media = MediaFileUpload(
    str(srt_path),
    mimetype="application/octet-stream",
    resumable=False,
)
try:
    youtube.captions().insert(
        part="snippet", body=body, media_body=media,
    ).execute()
except HttpError as exc:
    status = getattr(getattr(exc, "resp", None), "status", "?")
    # ``captions.insert`` requires the ``youtube.force-ssl`` OAuth
    # scope – most existing channel tokens were issued with only
    # ``youtube.upload`` and ``youtube`` (no force-ssl). Operator
    # caught this on TST Ep465 (May 6 2026): every long-form
    # upload logs HTTP 403 ``Insufficient Permission`` for the
    # caption track. Fix is to re-run the OAuth consent flow with
    # ``--scope https://www.googleapis.com/auth/youtube.force-ssl``
    # added (typically ``scripts/auth_youtube.py``) and replace
    # the channel refresh tokens in the GitHub Actions secrets.
    # The video upload itself still succeeds; this is best-effort.
    if status in (403, "403"):
        logger.warning(
            "Caption track upload rejected for %s (HTTP 403 – "
            "OAuth missing youtube.force-ssl scope). Re-run the "
            "channel auth flow with that scope added; the video "
            "upload itself succeeded so this is non-blocking. "
            "Underlying: %s",
            video_id, exc,
        )
    else:
        logger.warning(
            "Failed to upload caption track for %s (HTTP %s): %s",
            video_id, status, exc,
        )
    return False
except Exception as exc: # noqa: BLE001 – never crash the run
    logger.warning(
        "Caption track upload errored for %s: %s", video_id, exc,
    )
    return False
logger.info("Uploaded %s caption track for %s", language, video_id)

return True

```

3. Number of videos uploaded per day

Today the client uploads 4 videos per day. Only two shows are enabled for YouTube; each uploads one long-form video and one Shorts clip every day:

Show (channel)	Long-form	Shorts	Videos/day
Tesla Shorts Time (Nerra Network)	1	1	2
Models & Agents for Beginners (Nerra Network)	1	1	2
Total today	2	2	4

That is ~7,600 quota units/day — right against the 10,000-unit default ceiling, which is why only two shows can run today.

Per-episode quota cost

API call	Per episode	Units each	Subtotal
videos.insert (long-form + Shorts)	2	1,600	3,200
thumbnails.set	2	50	100
captions.insert	1	400	400
playlistItems.insert	2	50	100
Total per episode			≈ 3,800

At full rollout the network publishes 11 shows daily, each as a long-form + Shorts pair — up to **22 videos/day** (~41,800 units/day steady-state), plus temporary headroom to backfill ~450 historical Tesla episodes and the other shows' back-catalogs. This is the basis for the 190,000-unit additional quota request.

Where the uploads appear

Channels: youtube.com/@NerraNetwork (English) and youtube.com/@NerraRU (Russian). Recent public uploads produced by `videos.insert`: [Tesla Ep 479](#), [M&A Beginners Ep 45](#), [Tesla Ep 478](#). The screenshot exhibits that follow show these live on the channel, the scheduled run that performs the upload, and the synthetic-media disclosure on a published video.

Exhibit A — Automated daily uploads (YouTube Studio)

YouTube Studio → Content → Videos for the Nerra Network channel (May 21, 2026). Tesla Shorts Time and Models & Agents for Beginners each publish a new episode every day, all Public / Published. Every row is the output of a videos.insert API call.

Studio

Search across your channel

Create

N N

Your channel

Nerra Network

Dashboard

Content

Analytics

Community

Subtitles

Content detection

Earn

Customization

Audio library

Settings

Send feedback

Channel content

Inspiration

Videos

Shorts

Live

Posts

Playlists

Podcasts

Courses

Promotions

Collaborations

Filter

	Video	Visibility	Restrictions	Date ↓	Views	Comments	Likes (vs. dislikes)
☐	Recently, YU7 undercuts the Model Y by \$4,500 with extra range, raising the stakes for Tesla in China and beyond. 6:50 Tesla Shorts Time – Ep 479: Xiaomi's YU7 undercuts the Model Y by \$4,500 with extra range, raising the stakes for Tesla in China and beyond. Show page...	Public	None	May 21, 2026 Published	5	0	100.0% 2 likes
☐	An AI built for safer mental health chats just scored way higher on safety tests than anything else out there. 6:22 Models & Agents for Beginners – Ep 45: An AI built for safer mental health chats just scored way higher on safety tests than anything else out there. Show page...	Public	None	May 21, 2026 Published	4	0	-
☐	Tesla's decision to spotlight Signature Edition Model S and X comes as production focus shifts elsewhere in the lineup. 6:32 Tesla Shorts Time – Ep 478: Tesla's decision to spotlight Signature Edition Model S and X comes as production focus shifts elsewhere in the lineup. Show page...	Public	None	May 20, 2026 Published	37	0	-
☐	Google just turned AI from a chat buddy into something that can actually do your tasks for you. 6:41 Models & Agents for Beginners – Ep 44: Google just turned AI from a chat buddy into something that can actually do your tasks for you. Show page: https://nerranetwork.com/models-agents-	Public	None	May 20, 2026 Published	18	2	100.0% 1 like
☐	Tesla's scrapped India factory plans expose how regulatory and infrastructure hurdles can derail even aggressive global expansion goals. 6:51 Tesla Shorts Time – Ep 477: Tesla's scrapped India factory plans expose how regulatory and infrastructure hurdles can derail even aggressive global expansion goals. Show page...	Public	None	May 19, 2026 Published	34	0	100.0% 3 likes
☐	Your smart speaker can now write and record an entire podcast on any topic you name. 6:00 Models & Agents for Beginners – Ep 43: Your smart speaker can now write and record an entire podcast on any topic you name. Show page: https://nerranetwork.com/models-agents-	Public	None	May 19, 2026 Published	4	0	100.0% 1 like
☐	Tesla lost its top spot in Hungary to a Chinese rival, underscoring how quickly local market leadership can shift. 6:12 Tesla Shorts Time – Ep 476: Tesla lost its top spot in Hungary to a Chinese rival, underscoring how quickly local market leadership can shift. Show page...	Public	None	May 18, 2026 Published	25	0	-

Exhibit B — Scheduled run succeeded, including the upload job

GitHub Actions run #689 of the “Run Podcast Show” workflow. The run is green and the job list shows `run (tesla)` completed successfully — that job performs the `videos.insert` upload. The workflow is triggered on: `schedule (cron)`.

The screenshot displays the GitHub Actions interface for the repository `patricknovak/nerranetworks`. The top navigation bar includes links for Code, Issues, Pull requests, Actions (selected), Projects, Security and quality, and Insights. The main header shows the workflow name "Run Podcast Show" and the specific run "Run Podcast Show #689" with a green status icon. A "Sign in to view logs" button is visible in the top right.

On the left sidebar, the "Summary" section is active, showing a list of jobs: `gate`, `alert-skip`, `run (tesla)`, and `finalize`. The `run (tesla)` job is highlighted with a green checkmark. Below this, the "Run details" section is visible, with "Usage" and "Workflow file" tabs.

The "Workflow file" tab is selected, displaying the workflow file content for `github/workflows/run-show.yml`. The file is a YAML document defining the "Run Podcast Show" workflow. It includes a `name`, `on` trigger with a `schedule` block, and a `jobs` section with a `run` job. The `run` job uses the `tesla` action to perform an `insert` operation. The workflow file content is as follows:

```
1 name: Run Podcast Show
2
3 on:
4   schedule:
5     # Network ships ~10 episodes/day. All shows scheduled to finish before
6     # 8 AM Eastern (~12 UTC EDT / 13 UTC EST). Pipeline takes up to 45 min,
7     # so triggers must be staggered by at least that much.
8     #
9     # May 2026 schedule overhaul: 7 shows went daily (OV, PT, FF, MGA, MAB,
10    # MIT, TST) with Sunday's slot becoming a "weekly recap" episode that
11    # synthesizes the past 7 days from the content. Like MIT also runs
12    # Saturday with a weekend-focus angle (international markets, crypto,
13    # lessons learned). Cron times shifted +1h from the prior schedule.
14    #
15    # fpuwer, Pyccskål: Even days at 6:00 UTC
16    - cron: '0 6 2-31/2 * *'
17    # Omni View: Daily at 6:30 UTC (Sunday = weekly recap)
18    - cron: '30 6 * * *'
19    # Planetterrian Daily: Daily at 7:00 UTC (Sunday = weekly recap)
20    - cron: '0 7 * * *'
21    # Fascinating Frontiers: Daily at 7:30 UTC (Sunday = weekly recap)
22    - cron: '30 7 * * *'
23    # Environmental Intelligence: Odd weekdays at 8:00 UTC
24    - cron: '0 8 1-31/2 * 1-5'
25    # Models & Agents: Daily at 8:30 UTC (Sunday = weekly recap)
26    - cron: '30 8 * * *'
27    # Models & Agents for Beginners: Daily at 9:00 UTC (Sunday = weekly recap)
```

Exhibit C — Daily server-side cron schedule

The workflow file (`.github/workflows/run-show.yml`) showing the per-show daily cron triggers. The client runs unattended on this schedule — no human initiates each upload.

```
1  name: Run Podcast Show
2
3  on:
4    schedule:
5      # Network ships ~10 episodes/day. All shows scheduled to finish before
6      # 8 AM Eastern (~12 UTC EDT / 13 UTC EST). Pipeline takes up to 45 min,
7      # so triggers must be staggered by at least that much.
8      #
9      # May 2026 schedule overhaul: 7 shows went daily (OV, PT, FF, M&A, MAB,
10     # MIT, TST) with Sunday's slot becoming a "weekly recap" episode that
11     # synthesises the past 7 days from the content lake. MIT also runs
12     # Saturday with a weekend-focus angle (international markets, crypto,
13     # lessons learned). Cron times shifted +1h from the prior schedule.
14     #
15     # Привет, Русский!: Even days at 6:00 UTC
16     - cron: '0 6 2-31/2 * *'
17     # Omni View: Daily at 6:30 UTC (Sunday = weekly recap)
18     - cron: '30 6 * * *'
19     # Planetterrian Daily: Daily at 7:00 UTC (Sunday = weekly recap)
20     - cron: '0 7 * * *'
21     # Fascinating Frontiers: Daily at 7:30 UTC (Sunday = weekly recap)
22     - cron: '30 7 * * *'
23     # Environmental Intelligence: Odd weekdays at 8:00 UTC
24     - cron: '0 8 1-31/2 * 1-5'
25     # Models & Agents: Daily at 8:30 UTC (Sunday = weekly recap)
26     - cron: '30 8 * * *'
27     # Models & Agents for Beginners: Daily at 9:00 UTC (Sunday = weekly recap)
28     - cron: '0 9 * * *'
29     # Финансы Просто: Even days at 9:30 UTC
30     - cron: '30 9 2-31/2 * *'
31     # Modern Investing Techniques: Daily at 10:00 UTC.
32     # Mon-Fri = US market preview; Sat = weekend angle (international
33     # markets, crypto, lessons learned, what to prepare for, long-term
34     # insights); Sun = weekly recap.
35     - cron: '0 10 * * *'
36     # Tesla Shorts Time: Daily at 11:00 UTC (Sunday = weekly recap)
37     - cron: '0 11 * * *'
38     # Unintended Consequences: Weekdays at 11:30 UTC. Narrative show –
39     # topic-queue-driven, no daily news fetch. Mon-Fri only (5x/week).
40     - cron: '30 11 * * 1-5'
41
```

Exhibit D — Synthetic-media disclosure on a published video

Public watch page for Tesla Shorts Time Ep 479. The description carries a written AI disclosure, and YouTube's official "How this was made → Altered or synthetic content" notice is shown — set via `status.containsSyntheticMedia = true` on the upload.

Tesla Shorts Time — Ep 479: Xiaomi's YU7 undercuts the Model Y by \$4,350 with extra range, raisin.

**Nerra Network**
19 subscribers

Tesla Shorts Time — Ep 479: Xiaomi's YU7 undercuts the Model Y by \$4,350 with extra range, raisin...

6 views May 21, 2026 [Tesla Shorts Time Daily](#)

Xiaomi's YU7 undercuts the Model Y by \$4,350 with extra range, raising the stakes for Tesla in China and beyond.

Show page: <https://nerranetwork.com/tesla.html>

Subscribe to Tesla Shorts Time on the Nerra Network: <https://nerranetwork.com/tesla.html?u...>

Xiaomi's YU7 undercuts the Model Y by \$4,350 with extra range, raising the stakes for Tesla in China and beyond.

Today's Tesla Shorts Time episode (Ep 479, May 21, 2026) covers the stories above in full — listen on Spotify, Apple Podcasts, or any podcast app

Direct audio: <https://audio.nerranetwork.com/tesla/...>

Photos via Pexels:
Imagery generated by Grok Imagine (xAI).
Imagery generated by Grok Imagine (xAI).

AI Disclosure: This podcast is curated by Patrick but uses AI-generated voice synthesis (xAI Grok TTS) for the narration. Editorial selection, summaries, and analysis are written from primary sources by Nerra Network. Learn more: <https://nerranetwork.com/about>

—

Suggested pinned comment:

Full episode + show notes: {show_page_url}

Subscribe on your favourite podcast app — search "{show_name}" on the Nerra Network.

How this was made

Altered or synthetic content
Sound or visuals were significantly edited or digitally generated. [Learn more](#)

Ask